

02-1 가상 DOM 이해하기

React 렌더링과 화면 변경 관찰

버튼을 누르면 숫자만 바뀌는 이유는 무엇일까요?

카운터 예제로 상태 변경과 DOM 수정을 살펴봅시다.

React

DOM과 가상 DOM

화면 구조를 표현하는 두 가지 방식

DOM: 브라우저가 관리하는 실제 문서 구조

가상 DOM: React가 메모리에 유지하는 UI 표현을 가리키는 개념

JSX로 작성한 화면 설명을 바탕으로 React가 변경 사항을 계산합니다.

개발자 도구의 Elements 탭은 실제 DOM을 보여줍니다.

상태 변경부터 화면 반영까지

이번 예제에서 버튼을 한 번 눌렀을 때

1. 이벤트: 버튼의 onClick 함수가 실행됩니다.
2. 상태 변경 요청: setCount가 다음 숫자를 계산하도록 요청합니다.
3. 렌더링: App 함수를 실행해 다음 UI를 계산하고 이전 결과와 비교합니다.
4. 커밋: 실제 DOM에 필요한 변경을 적용합니다.

이후 브라우저가 변경된 화면을 그립니다.

다시 렌더링해도 DOM은 유지됩니다

이 카운터에서는 숫자 텍스트가 달라집니다.

변경 전: `<p class="count">0</p>`

변경 후: `<p class="count">1</p>`

App 함수가 다시 실행되어도 제목과 버튼 DOM을 매번 새로 만들지 않습니다.

렌더링은 UI 계산, 커밋은 실제 DOM에 변경을 적용하는 단계입니다.

실습 파일 준비

기존 hello-app 프로젝트를 사용합니다.

VS Code에서 hello-app 폴더를 엽니다.

src/App.jsx: 카운터의 화면과 동작

src/App.css: 남색 배경과 하늘색 디자인

src/main.jsx: React 앱을 실제 DOM에 연결

각 파일의 기존 내용을 교체합니다. 분할된 코드는 순서대로 이어 붙입니다.

App.jsx (1/2)

상태 선언과 화면 구성

```
import { useState } from "react";
import "./App.css";

export default function App() {
  const [count, setCount] = useState(0);
  console.log("App 렌더링:", count);

  return (
    <main className="counter-page">
      <section className="counter">
        <h1>React 렌더링 실습<span>.</span></h1>
        <p className="label">클릭 횟수</p>
        <p className="count" aria-live="polite">
          {count}
        </p>
      </section>
    </main>
  );
}
```

다음 장의 코드를 바로 이어서 입력합니다.

App.jsx (2/2)

버튼 클릭으로 상태 변경 요청

```
<button
  type="button"
  onClick={() => setCount((prev) => prev + 1)}
>
  + 1 증가
</button>
<p className="description">
  버튼을 눌러 숫자의 변화를 확인하세요.
</p>
</section>
</main>
);
}
```

prev는 이전 상태 값입니다. 이전 값에 1을 더해 갱신합니다.

App.css (1/4)

기본 여백과 화면 중앙 정렬

```
* {  
  box-sizing: border-box;  
}  
body {  
  margin: 0;  
  font-family: Arial, "맑은 고딕", sans-serif;  
  background: #101827;  
  color: white;  
}  
.counter-page {  
  min-height: 100svh;  
  display: grid;  
  place-items: center;  
  padding: 32px 20px;
```

이어서 배경을 설정하고 .counter-page의 종괄호를 닫습니다.

App.css (2/4)

배경과 제목 디자인

```
background: radial-gradient(
  ellipse at center, #1b3045 0%, #101827 70%
);
}
.counter {
  width: 100%;
  max-width: 720px;
  text-align: center;
}
.counter h1 {
  margin: 0 0 48px;
  font-size: clamp(28px, 6vw, 52px);
  line-height: 1.3;
  letter-spacing: -0.04em;
}
.counter h1 span { color: #61dafb; }
```

App.css (3/4)

클릭 횟수와 숫자 디자인

```
.label {  
  margin: 0;  
  color: #b7c6d9;  
  font-size: 20px;  
}  
.count {  
  margin: 12px 0 32px;  
  color: #61dafb;  
  font-size: clamp(80px, 20vw, 140px);  
  font-weight: bold;  
  line-height: 1.2;  
  overflow-wrap: anywhere;  
}
```

clamp()로 화면 크기에 따라 글자 크기를 조절합니다.

App.css (4/4)

버튼과 안내 문구 디자인

```
.counter button {  
  padding: 16px 36px;  
  border: none;  
  border-radius: 12px;  
  background: #61dafb;  
  color: #101827;  
  font: inherit;  
  font-size: 20px;  
  font-weight: bold;  
  cursor: pointer;  
  transition: background 0.2s;  
}  
.counter button:hover {  
  background: #a0eaff;  
}
```

```
.counter button:active {  
  background: #38bfe5;  
}  
.counter button:focus-visible {  
  outline: 3px solid white;  
  outline-offset: 5px;  
}  
.description {  
  margin: 28px 0 0;  
  color: #b7c6d9;  
  font-size: 16px;  
  line-height: 1.6;  
}
```

main.jsx와 실행 확인

기본 index.css는 불러오지 않습니다.

```
import React from "react";  
import ReactDOM from "react-dom/client";  
import App from "./App.jsx";
```

```
ReactDOM.createRoot(  
  document.getElementById("root")  
)  
.render(  
  <React.StrictMode>  
    <App />  
  </React.StrictMode>  
);
```

세 파일 저장 후 확인합니다. 서버가 꺼져 있으면 hello-app에서 npm run dev를 실행합니다.

Console에서 렌더링 관찰

F12로 개발자 도구를 열고 Console 탭을 선택합니다.

1. 화면의 + 1 증가 버튼을 누릅니다.
2. App 렌더링: 1, App 렌더링: 2 출력을 확인합니다.

이 로그는 App 함수가 실행됐다는 뜻입니다.

개발 모드의 StrictMode는 검사 목적으로 함수를 추가 실행할 수 있습니다.

로그가 두 번 보여도 클릭 횟수가 두 번 증가한다는 뜻은 아닙니다.

Elements에서 실제 DOM 관찰

Elements 탭에서 class가 count인 p 요소를 찾습니다.

1. 요소 선택 도구로 화면의 큰 숫자를 선택합니다.
2. `<p class="count" ...>0</p>`를 확인합니다.
3. + 1 증가 버튼을 누르고 숫자 텍스트의 변화를 관찰합니다.

제목과 버튼의 내용은 그대로 유지됩니다.

Elements는 실제 DOM입니다. 가상 DOM 자체를 보여주는 탭은 아닙니다.

실습에서 확인한 것

가상 DOM의 역할과 직접 관찰한 대상을 구분합니다.

React 내부: 다음 UI를 계산하고 이전 결과와 비교

Console: 컴포넌트 함수의 실행 기록

Elements: 실제 DOM에 반영된 변경

브라우저 화면: 변경된 숫자

확인 질문: App이 다시 실행되면 제목 DOM도 반드시 새로 만들어질까요?